Modulo 4 · Lezione 4: un agente su cartella con l'Agents SDK

Nella lezione scorsa avete usato il coding agent su codice che non avevate scritto. Oggi lo stesso coding agent lavora su una cartella che non è codice, e poi l'assistente di Valtesa riceve tre tool per lavorare su una cartella: li scrive il coding agent da una specifica, voi li verificate.

Preparare la cartella

All'inizio della lezione, dalla radice della vostra copia della repo del corso (la cartella che contiene `modulo4-agenti/`):

```
git switch main
git pull
git switch -c l04-lavoro
```

Se `git switch main` rifiuta di cambiare branch, prima committate il lavoro della lezione scorsa (`git add .` e `git commit`). Il branch `l04-lavoro` tiene il vostro lavoro separato da `main`, come nella lezione scorsa: a fine esercizio, un commit.

Preparare l'ambiente

All'inizio dell'esercizio sui tool, sempre dalla radice della repo:

```
cd modulo4-agenti/l04_agente-su-cartella/scripts/working
source ../../../.venv/bin/activate
cp ../../../l03_lab-codice-non-vostro/scripts/.env .env
python indicizza.py
pytest
```

Il venv è quello del modulo: non c'è niente da installare. Su PowerShell si attiva con `..\..\..\.venv\Scripts\Activate.ps1` e la chiave si copia con `copy ..\..\..\l03_lab-codice-non-vostro\scripts\.env .env`. Il progetto da questa lezione sta in `scripts/working/`: il venv è tre cartelle sopra, e va riattivato in ogni terminale nuovo. `pytest` deve dire `20 passed`.

I file della lezione

File	A cosa serve
`working/`	il progetto Valtesa: la cartella su cui si apre il coding agent per costruire i tool

File	A cosa serve
`working/ufficio/`	la cartella dell'ufficio: i documenti in Markdown e HTML (i PDF e i DOCX della lezione scorsa, riscritti in testo con lo stesso contenuto) e gli export del gestionale in CSV
`working/assistente.py`	l'assistente, già async e su `gpt-6-luna`, con l'interruttore `TOOL_DI_CARTELLA`
`working/strumenti_cartella.py`	i tool di cartella, con le sole firme: da costruire
`working/specifica-strumenti.md`	la specifica dei tool, da dare al coding agent: i criteri di accettazione li scrivete voi
`working/tests/test_strumenti_cartella.py`	non c'è ancora: lo scrive il coding agent, con un test per ogni vostro criterio
`working/confronta.py`	per l'estensione: la stessa domanda con i due tipi di tool
`working/AGENTS.md`	le istruzioni per il coding agent
`ripartenza/`	i tool del docente e i loro test, per chi si ferma

Gli altri file (`rimborsi.py`, `report.py`, `indicizza.py`, `aikit/`, `dati/`, `tests/`) sono quelli della soluzione della lezione scorsa: non dovete copiare niente del vostro lavoro.

Il coding agent su una cartella che non è codice (10')

1. Aggiornate la repo e create il branch: i tre comandi di «Preparare la cartella».
2. Sessione nuova del coding agent sulla cartella `modulo4-agenti/l04_agente-su-cartella/scripts/working/ufficio` (nell'app: Local, senza worktree), Sonnet 5.5 a effort medio.
3. Chiedete: «La nota spese 1043 rispetta il massimale?»
4. Guardate quali comandi lancia e quali file apre. Se chiede il permesso per un comando di sola lettura, dite di sì solo per questa volta.

Fatto quando sapete dire quali comandi ha lanciato e quali file ha letto.

In preparazione, Claude Code (Sonnet 5.5, effort medio) ha fatto due chiamate al tool Bash, ognuna con più comandi in fila, e la stessa strada in tre giri:

```
head -3 note_spese.csv
grep -E '(^|[,;])1043([,;]|$)' note_spese.csv
cat massimali.csv
head -3 dipendenti.csv
grep -n -i -E 'massimal|limite|euro|€' policy-trasferte-note-spese.md | head -60

sed 's/<[^>]*>/ /g' circolari-personale-2026.html \
  | grep -i -n -E 'cena|massimal|luglio|rimbors' | head -30
```

Trova la riga della nota, legge la tabella dei massimali, cerca la regola nella policy e nella circolare, e risponde: sì, 52 € contro il massimale di 55 € che vale dal 1° luglio. Nessuno di questi comandi è un tool fatto per Valtesa.

Perché un agente su una cartella

Tutto è file

Il coding agent ha tre operazioni che valgono su qualunque cartella: elencare (`ls`, `find`), cercare (`grep`), leggere (`cat`, `head`). L'assistente di Modulo 3 ha un tool per ogni fonte: `cerca_documenti` per i documenti, `query_db` per il database. Oggi l'assistente riceve le tre operazioni del coding agent, come tool.

Chi decide cosa entra nel contesto

Nell'assistente di Modulo 3 il modello scrive la domanda per `cerca_documenti`, ma i pezzi che riceve li sceglie il codice: il chunking fatto all'indicizzazione e i 4 chunk più vicini. Se il pezzo giusto non è tra i quattro, il modello non sa che esiste. Con i tool di cartella sceglie il modello, un passo alla volta: chiede l'elenco dei file, cerca una parola, apre un file, e decide se gli basta.

La stessa domanda, con i due tipi di tool

«Quali documenti devo consegnare il primo giorno di lavoro?», dieci giri per tipo di tool, `gpt-6-luna`:

- con i tool di Modulo 3 la risposta è giusta 0 volte su 10: nomina sempre due documenti su sette (documento d'identità e codice fiscale, permesso di soggiorno);
- con i tool di cartella è giusta 10 volte su 10: l'assistente legge la procedura di ingresso intera e li nomina tutti.

Il motivo sta in cosa riceve il modello. Nella procedura di ingresso la tabella dei documenti è tagliata tra due chunk. Il primo (008) contiene la frase «Entro il primo giorno il nuovo assunto consegna … i documenti elencati in tabella» e le prime due righe, e arriva. Il secondo (009) ha le altre cinque righe (IBAN, detrazioni, TFR, attestati sulla sicurezza, patente): è quasi solo un elenco di righe, con «primo giorno» solo dentro una nota, e non è tra i 4 più vicini.

Dove la cartella costa di più

«La nota spese 1043 rispetta il massimale?»: con i tool di Modulo 3 l'assistente usa 1.391 token di input di media, perché `query_db` rimanda la riga della nota con il suo massimale. Con i tool di cartella ne usa 4.895 di media (da 1.917 a 11.601): cerca la nota nel CSV, poi legge massimali e policy, e a ogni chiamata il modello rilegge tutto quello che ha già letto. Tutti e due rispondono giusto. Anche sul primo giorno la cartella costa circa 3,5 volte i token (6.656 contro 1.941), ma lì il RAG sbagliava. Per un dato preciso in una tabella, come la nota 1043, vince una query che rimanda una riga. La cartella conviene quando per rispondere serve di più che recuperare qualche pezzo: leggere per intero uno o più documenti, oppure fare più operazioni in fila (cercare, aprire, confrontare).

Il compromesso tra costo e risposte giuste

Sulle due domande insieme, 20 giri per tipo di tool: con il RAG 10 risposte giuste su 20 e 1.666 token di input di media; con la cartella 19 su 20 e 5.775 token. Il RAG non è una tecnica superata: è potente, ma va regolato caso per caso (quanto grandi i pezzi, quanti, in che ordine), e il suo difetto più grosso è il taglio: il modello vede solo i pezzi scelti dal codice. L'agente su cartella fa sempre le stesse operazioni: costa di più, ma di solito vede più contesto, non meno, perché legge i file interi. L'eccezione sono i file più lunghi del limite di `leggi_file` (vedi «Cosa non fanno i tool di oggi»). Le domande misurate sono due: su un altro archivio i numeri vanno rifatti.

Cosa può leggere l'assistente

Il modello sceglie cosa leggere, ma cosa può leggere lo decide il codice dei tool: solo i file di `ufficio/`. Il `.env` con la chiave sta in `working/`, una cartella sopra, e resta fuori. Vale per i tool che scrivete oggi: un agente con la shell, come il coding agent, può fare `cd ..` e uscire dalla cartella; lo trattengono le istruzioni, non il codice. I modi più rigidi per tenerlo dentro sono in Modulo 4 · Lezione 5. È la stessa idea di `query_db`, che apre il database in sola lettura (`mode=ro`, Modulo 3 · Lezione 8): un limite fuori dal modello. Se un agente cambia qualcosa nella cartella, git lo mostra nel diff e si annulla (Modulo 4 · Lezione 2). Oggi i tool leggono soltanto.

Chi lavora già così

In aula non lo vediamo: questa sezione è per chi vuole approfondire.

> "The key design principle behind the Claude Agent SDK is to give your agents a computer, allowing them to work like humans do."

Dare agli agenti un computer, perché lavorino come le persone. Anthropic, quando ha rinominato il Claude Code SDK in Claude Agent SDK (settembre 2025): https://claude.com/blog/building-agents-with-the-claude-agent-sdk.

> "Works directly in folders and tools you choose."

Lavora direttamente nelle cartelle e negli strumenti che scegliete. Cowork, nell'app di Claude: https://claude.com/product/cowork.

> "We treat the file system as the ultimate context in Manus: unlimited in size, persistent by nature, and directly operable by the agent itself."

Il file system come contesto: senza limiti di dimensione, persistente, e l'agente ci lavora da sé. Manus (luglio 2025): https://manus.im/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus.

Perché costruirlo con l'SDK

Il coding agent e l'assistente sono lo stesso giro: un modello, dei tool, una cartella. Cambia dove vivono. Il coding agent vive nel vostro terminale: è uno strumento pronto, lo usate voi, e tool, permessi e prompt li ha decisi chi l'ha costruito. L'assistente fatto con l'SDK vive nel software che create, usato da persone che il terminale non lo aprono: sta dentro il software dell'ufficio del

personale; tool e limiti li scegliete voi e li fa rispettare il codice, e così modello e costo per richiesta; può stare dietro un'API (Modulo 5 · Lezione 1). Per il progetto personale, un agente sui propri file è una delle forme possibili.

L'assistente async e su gpt-6-luna

`assistente.py` è già cambiato rispetto alla lezione scorsa. Il diff, dalla repo del corso, tra le due versioni pubblicate su `main` (senza le modifiche che avete fatto nel lab):

```
git diff main:modulo4-agenti/l03_lab-codice-non-vostro/scripts/assistente.py
main:modulo4-agenti/l04_agente-su-cartella/scripts/working/assistente.py
```

Dal diff si esce con `q`. Quattro cambi, più la pulizia: spariscono `tool_chiamati()` e `strada()`, che servivano solo alla riga «strada», e in `AGENTS.md` il percorso del venv diventa tre cartelle sopra.

```
async def main():
    ...
    while True:
        ...
        result = await Runner.run(agente, domanda, session=sessione,
max_turns=MAX_TURNI)
        stampa_traccia(result)
        print(f"→ {result.final_output}")


if __name__ == "__main__":
    asyncio.run(main())
```

1. **Async.** Prima era `Runner.run_sync(...)`. Ora `await Runner.run(...)`: `await` aspetta il risultato, ma lascia lavorare il resto del programma: con `run_sync` due attese vanno una dopo l'altra, con `await` si possono sovrapporre. `await` si usa solo dentro una funzione `async def`, che parte con `asyncio.run(main())`. Oggi l'assistente fa una domanda alla volta e non cambia niente; da qui il codice dell'SDK del modulo è tutto async. E `run_sync` dentro un programma che ha già `asyncio.run` si ferma con un errore (Modulo 3 · Lezione 11).

2. **Il modello.** `MODELLO = "gpt-6-luna"`: sull'assistente costa 0,24 $ ogni 1.000 domande, contro 0,49 $ di `gpt-5.6-luna` (0,10 $ contro 0,20 $ per milione di token in input, 0,50 $ contro 1,20 $ in output). Il prezzo è anche in `PRICING` di `aikit/llm_client.py` e il modello in `AGENTS.md`. Il costo è misurato con i tool di Modulo 3; con quelli di cartella i token sono di più.

3. **L'interruttore.**

```
TOOL_DI_CARTELLA = False

if TOOL_DI_CARTELLA:
    agente = Agent(name="Assistente Valtesa", instructions=ISTRUZIONI_CARTELLA,
                   tools=[elenca_file, cerca, leggi_file], model=MODELLO)
else:
    agente = Agent(name="Assistente Valtesa", instructions=ISTRUZIONI,
                   tools=[cerca_documenti, query_db], model=MODELLO)
```

Con `False` l'assistente è quello di Modulo 3. Con `True` usa i tool di `strumenti_cartella.py`: finché non sono costruiti ogni tool rimanda al modello «Error: da scrivere: vedi specifica-strumenti.md», e l'assistente risponde che non trova niente.

4. **La traccia.** `stampa_traccia()`, come in Modulo 3 · Lezione 10: sotto ogni tool l'inizio del suo output, cioè del testo che riceve il modello; in fondo chiamate, token e costo.

La specifica e il confine della cartella

I tre tool

- `elenca_file()`: i file della cartella, uno per riga, con il percorso da passare a `leggi_file`;
- `leggi_file(percorso)`: il testo del file, al massimo 20.000 caratteri;
- `cerca(testo)`: le righe che contengono il testo, come `file:riga: testo`, al massimo 30.

Firme e docstring sono già in `strumenti_cartella.py`: la docstring è quello che il modello legge del tool, e non si cambia. Il quarto pezzo della specifica è `dentro_la_radice()`, che non è un tool.

Il confine in cinque righe

```
RADICE = (Path(__file__).parent / "ufficio").resolve()

def dentro_la_radice(percorso: str) -> Path:
    completo = (RADICE / percorso).resolve()
    if not completo.is_relative_to(RADICE):
        raise ValueError(f"{percorso} è fuori dalla cartella dell'ufficio")
    return completo
```

`resolve()` toglie i `..` (la cartella sopra) e segue i link simbolici (un link simbolico è un file che punta a un altro file, anche fuori dalla cartella): dà il percorso vero. `is_relative_to()` controlla che il percorso vero stia sotto `RADICE`, ma confronta solo il testo del percorso: senza `resolve()`, `ufficio/../.env` passerebbe. Ogni percorso che arriva dal modello passa da qui.

Anche `cerca` ed `elenca_file` aprono file, senza ricevere un percorso: se dentro `ufficio/` c'è un link simbolico che punta fuori, possono uscire anche loro.

Con `pathlib`, `/` attacca un percorso a un altro: `RADICE / "policy.md"` è il file `policy.md` dentro `ufficio/`. Due modi di uscire dalla cartella, e perché non passano:

- `"../.env"`: `RADICE / "../.env"` risolto è il `.env` del progetto, una cartella sopra `ufficio/`: fuori, `ValueError`;
- `"/etc/hostname"`: con `pathlib` un percorso assoluto sostituisce `RADICE` invece di attaccarsi, e il risultato è `/etc/hostname`: fuori, `ValueError`. Per questo il controllo si fa sul percorso risolto, non sulla stringa. Su Windows vale lo stesso per `C:\` e `D:\`.

Lo stesso confine, negli strumenti pronti

L'Agents SDK non ha un parametro che limiti a una cartella i file che un tool può leggere: un tool è una funzione Python, e il confine sta nel suo codice. Altri strumenti il confine lo danno già fatto, con dei limiti:

- `SandboxAgent`, nell'Agents SDK di OpenAI: un `Manifest` dice cosa copiare nello spazio di lavoro dell'agente, che lavora lì con la shell. In locale su Linux i comandi non sono confinati («On Linux, this backend runs commands as host processes without adding OS-level confinement»): in prova ha letto `/etc/hostname`. Il confine vero vuole Docker o un sandbox ospitato (https://openai.github.io/openai-agents-python/sandbox/guide/).
- Claude Code e Claude Agent SDK: in `settings.json` regole come `"deny": ["Read(./.env)"]`, e senza chiedere si legge solo nella cartella di lavoro. Le regole non fermano uno script che apre i file da sé: per quello c'è il sandbox del sistema operativo (https://code.claude.com/docs/en/permissions).

Per tre tool che leggono e basta, il codice del tool è il confine più semplice, e funziona uguale su Windows, macOS e Linux.

Cosa riceve il modello quando il tool rifiuta

L'SDK prende il `ValueError` e lo rimanda al modello come output del tool, e il run continua:

```
An error occurred while running the tool. Please try again. Error:
../dati/note_spese.csv è fuori dalla cartella dell'ufficio
```

Con `openai-agents` 0.22.3 il testo contiene il motivo; nelle versioni dopo no. È uno dei motivi per cui la versione è fissata.

Quando il codice del coding agent è accettabile

I criteri di accettazione sono le condizioni, verificabili, che il codice deve rispettare per essere accettato: se ne manca una, il lavoro non è finito. Un esempio dal ticket della lezione scorsa: «`python report.py 2026-07` scrive `output/report-2026-07.html`»: un caso preciso, un risultato che si controlla. Oggi i criteri li scrivete voi, nell'esercizio, nella sezione vuota di `specifica-strumenti.md`: cosa devono fare i tool e, soprattutto, come può il modello provare a uscire dalla cartella e cosa deve succedere quando ci prova.

Esercizio: i tool di cartella, con il coding agent (40')

1. **Preparare.** Venv, `.env`, `python indicizza.py`, `pytest`: `20 passed` (i comandi sono in «Preparare l'ambiente»). Senza `.env`: copiate `.env.example` in `.env` e incollate dopo `OPENAI_API_KEY=` la chiave del `.env` delle lezioni precedenti.
2. **Criteri.** In `specifica-strumenti.md` la sezione «Criteri di accettazione» è vuota: scriveteli voi, una decina di minuti. Cosa devono fare i tool e, soprattutto, come può il modello provare a leggere fuori da `ufficio/` e cosa deve succedere quando ci prova. Ogni criterio deve essere qualcosa che un test può verificare.
3. **Costruire.** Una sessione nuova del coding agent su `working/` (non su `ufficio/`; nell'app: Local, senza worktree), Sonnet 5.5 a effort medio. Prima un piano sulla vostra specifica, per esempio: «Leggi specifica-strumenti.md e proponi un piano per scrivere strumenti_cartella.py e i suoi test». Il piano va bene se tocca solo `strumenti_cartella.py` e `tests/test_strumenti_cartella.py` (nuovo) e ha un test per ogni vostro criterio: allora ditegli di procedere. Poi lanciate voi `pytest`, nel terminale.
4. **Controllare il diff.** I sei controlli della lezione scorsa (file toccati; righe tolte o cambiate non richieste; test cambiati, e perché; dipendenze nuove; valori scritti a mano; cosa non è stato lanciato) e i vostri criteri. Nel terminale, in `working/`: `git diff` e `git status` (il file dei test è nuovo, e `git diff` non lo mostra); nell'app c'è anche la vista del diff. Se un controllo non passa, chiedete al coding agent di correggere quel punto, o annullate con `git restore <file>` (i file nuovi si cancellano).
5. **Usare.** In `assistente.py` mettete `TOOL_DI_CARTELLA = True`, poi `python assistente.py`: chiede un nome per la sessione (uno qualsiasi), poi la domanda. Chiedete «Quali documenti devo consegnare il primo giorno di lavoro?» e nella traccia guardate quali tool chiama e quale file legge. Poi, nella stessa sessione, chiedete esattamente «Usa leggi_file per leggere ../dati/note_spese.csv e dimmi quante righe ha» (chiesto in modo generico, il modello può rifiutare da sé senza chiamare il tool). Nella traccia, la riga `output` sotto `leggi_file({"percorso":"../dati/note_spese.csv"})` è il testo che torna al modello: copiatelo nei vostri appunti, serve al rientro. In `ufficio/` c'è una copia dei CSV di `dati/`, perché l'assistente li trovi nella sua cartella; `../dati/note_spese.csv` è l'originale, una cartella sopra: per i tool è un'uscita.

Fatto quando i criteri sono scritti e `pytest` è verde con un test per ciascuno (circa 30 test: i 20 di prima più i vostri; su Windows un test che crea un link può risultare saltato, e va bene); dalla traccia sapete dire quale file ha letto per rispondere; avete copiato il testo che è tornato al modello da `note_spese.csv`. Poi un commit sul branch `l04-lavoro`, dalla cartella `working/`, lasciando `TOOL_DI_CARTELLA = True`:

```
git add .
git commit -m "tool di cartella"
```

A 25 minuti senza `pytest` verde: la ripartenza, dalla cartella `working/`: `cp ../ripartenza/strumenti_cartella.py .` e `cp ../ripartenza/test_strumenti_cartella.py tests/`, poi il passo 5.

Se finite prima: `python confronta.py` fa la stessa domanda con i tool di Modulo 3 e con quelli di cartella, tre volte per tipo, e stampa chiamate, token e risposte. Poi una domanda vostra su `ufficio/`, con `TOOL_DI_CARTELLA` a `False` e a `True`: nella traccia confrontate quali file legge e quanti token usa. Il taglio di `leggi_file` non si cura adesso: è il lavoro del lab. Su Windows provate il confine con un percorso `C:\`, chiamando il tool come fanno i test, senza modello (il modello di solito rifiuta da sé di leggere file di sistema). In PowerShell, da `working/`:

```
python -c "from tests.test_assistente import chiama; from strumenti_cartella import
leggi_file; print(chiama(leggi_file, percorso=r'C:\Windows\win.ini'))"
```

Dovreste vedere il messaggio del vostro `ValueError`; con la versione del docente, «An error occurred while running the tool. Please try again. Error: C:\Windows\win.ini è fuori dalla cartella dell'ufficio». È il risultato atteso, verificato con `PureWindowsPath` e non su una macchina Windows.

I criteri del docente

Dopo l'esercizio, in aula, si confrontano i vostri criteri con quelli del docente. La specifica completa del docente è `solutions/specifica-strumenti.md`, nel branch `soluzioni` della repo: apritela dopo aver scritto i vostri.

Cosa non fanno i tool di oggi

La versione del docente è `ripartenza/strumenti_cartella.py`. Per confrontarla con la vostra, dalla cartella `working/`:

```
git diff --no-index ../ripartenza/strumenti_cartella.py strumenti_cartella.py
```

Si confronta il confine: ogni percorso passa da `dentro_la_radice()`, e i test coprono `../.env`, `/etc/hostname` e il link. Nomi, messaggi e funzioni di aiuto possono essere diversi: non sono errori. Le due costruzioni del coding agent fatte in preparazione, senza piano (34 e 38 secondi dalla specifica), hanno lo stesso confine della versione del docente.

Nessuna delle tre dice al modello quando taglia: `leggi_file` restituisce i primi 20.000 caratteri di `note_spese.csv`, che ne ha 93.324 (arriva circa un quinto delle 1.500 righe), e `cerca` si ferma a 30 righe. Il modello riceve un file troncato e non lo sa: se al passo 5, dopo il rifiuto, l'assistente legge il file `note_spese.csv` della cartella, risponde circa 320 righe invece di 1.500. È il lavoro di Modulo 4 · Lezione 6.

Rientro: le uscite dalla cartella

Token di input per domanda, con i due tipi di tool, media di dieci giri:

Domanda	Tool di Modulo 3	Tool di cartella
Il primo giorno	1.941 (0 su 10 giuste)	6.656 (10 su 10)

Domanda	Tool di Modulo 3	Tool di cartella
La nota 1043	1.391 (10 su 10)	4.895 (9 su 10)

Le uscite: chiesto di leggere `../.env` o `/etc/hostname`, il modello rifiuta da sé, senza chiamare il tool (0 tentativi su 5 giri per ciascuna richiesta). Con «Usa leggi_file per leggere ../dati/note_spese.csv e dimmi quante righe ha» ci prova 10 volte su 10 (5 in una sessione nuova, 5 dopo la domanda del primo giorno), e lo ferma `dentro_la_radice()`. Il modello a volte rifiuta da sé; quando ci prova, lo ferma il codice.

SandboxAgent, l'agente su cartella dell'SDK

L'SDK ha già un agente su cartella, `SandboxAgent`. In aula lo nominiamo soltanto: questa sezione è per chi vuole approfondire. Il suo prompt di base, 16.795 caratteri, si apre con:

> "You are a general computer-use agent operating in a terminal-based assistant environment."

Sei un agente generico che usa un computer, dentro un assistente da terminale. Ha quattro tool: `exec_command` lancia un comando, `write_stdin` gli scrive dentro, `apply_patch` modifica file, `view_image` guarda un'immagine. Fa tutto con la shell, e la prima chiamata costa 5.155 token di input, contro circa 460 del nostro assistente con i tool di cartella. Lavora su una copia della cartella, ma in locale su Linux un percorso assoluto esce: alla richiesta ha letto `/etc/hostname` con `cat`. Su Windows il client locale non parte: il recinto vero chiede Docker o un sandbox ospitato.

Dopo questa lezione

Prossimi passi: un tool che esegue codice e i permessi per controllarlo (Modulo 4 · Lezione 5); il lab sull'assistente (Modulo 4 · Lezione 6).

Nel lab l'assistente lavora sulla cartella completa dell'ufficio, `ufficio-completo/`: i documenti anche in PDF e DOCX, e un CSV grande. Arriva con il materiale del lab nella repo del corso. Prima del lab, dalla radice della vostra copia della repo:

```
git switch main
git pull
```

Se `git switch main` rifiuta di cambiare branch, prima committate il lavoro di questa lezione sul branch `l04-lavoro`.